

## **PATERNI PONAŠANJA**

### **STRATEGY PATTERN**

Najznačajniji obrazac ponašanja koji je već prisutan u sistemu jeste Strategy obrazac. U okviru modula za plaćanje definisan je interfejs StrategijaPlacanja, dok klase GotovinskoPlacanje, KarticnoPlacanje i LoyaltyPlacanje predstavljaju konkretne strategije. Na ovaj način objekat Placanje ne mora poznavati detalje implementacije svakog načina plaćanja, već koristi odgovarajuću strategiju u zavisnosti od izbora korisnika. Prednost ovakvog pristupa je jednostavno proširivanje sistema novim metodama plaćanja bez izmjena postojećeg koda, čime se poštuje Open-Closed princip.

### **STATE PATTERN**

Veoma pogodna primjena u ovom sistemu bila bi implementacija State obrasca za upravljanje životnim ciklusom sesije igranja. Klasa Sesija trenutno koristi enumeraciju StatusSesije sa vrijednostima Aktivna, Pauzirana, Završena i Istekla. Međutim, kako broj pravila vezanih za pojedina stanja raste, javlja se potreba za odvajanjem ponašanja svakog stanja u zasebne klase. Aktivna sesija omogućava obračun vremena i pokretanje pauze, pauzirana sesija dozvoljava nastavak rada, dok završena i istekla sesija ne dozvoljavaju dalje izmjene. Primjenom State obrasca svako stanje bi bilo predstavljeno zasebnom klasom, čime bi se izbjegli brojni uslovni izrazi i olakšalo održavanje sistema.

### **OBSERVER PATTERN**

Observer obrazac mogao bi se primijeniti u dijelu sistema koji se odnosi na takmičenja i loyalty program. Korisnici bi mogli biti pretplaćeni na određene događaje u sistemu. Kada se kreira novo takmičenje, promijeni termin održavanja ili otvore prijave za učešće, svi zainteresovani korisnici automatski bi bili obaviješteni. Sličan princip može se koristiti i u loyalty programu, gdje bi korisnik bio obaviješten kada ostvari određeni broj bodova ili stekne pravo na pogodnosti. Na taj način postiže se automatska komunikacija između objekata bez njihovog međusobnog čvrstog povezivanja.

### **COMMAND PATTERN**

U sistemu postoji veliki broj korisničkih akcija kao što su kreiranje rezervacije, otkazivanje rezervacije, pokretanje sesije, pauziranje sesije, završavanje sesije ili prijavljivanje kvara na uređaju. Svaka od ovih akcija može se predstaviti kao zaseban objekat komande. Prednost ovakvog pristupa ogleda se u mogućnosti evidentiranja svih izvršenih akcija, implementacije historije aktivnosti, kao i eventualne podrške za Undo i Redo funkcionalnosti.

## **CHAIN OF RESPONSIBILITY PATTERN**

Chain of Responsibility obrazac mogao bi se koristiti prilikom obrade zahtjeva za rezervaciju uređaja. Prije nego što rezervacija bude uspješno kreirana, potrebno je izvršiti više provjera. Sistem mora utvrditi da li korisnik postoji i da li je aktivan, da li je odabrani uređaj dostupan, da li uređaj nije u kvaru te da li korisnik ispunjava sve potrebne uslove za rezervaciju. Umjesto implementacije svih provjera unutar jedne metode, svaka provjera može biti predstavljena posebnim objektom koji obrađuje zahtjev i po potrebi ga prosljeđuje narednom objektu u lancu. Takav pristup omogućava jednostavno dodavanje novih pravila bez izmjena postojećeg koda.

## **TEMPLATE METHOD PATTERN**

Template Method obrazac mogao bi se primijeniti kod obračuna cijene korištenja uređaja. Bez obzira da li korisnik koristi PC, PlayStation ili Xbox, proces obračuna uglavnom prati isti slijed koraka: određivanje trajanja sesije, izračunavanje osnovne cijene, primjena eventualnih popusta i određivanje konačnog iznosa za naplatu. Opšti algoritam mogao bi biti definisan u nadklasi Uredaj, dok bi pojedine vrste uređaja redefinisale samo specifične dijelove obračuna.

## **ITERATOR PATTERN**

Iterator obrazac bio bi koristan za rad sa kolekcijama objekata unutar sistema. Administrator često ima potrebu pregledati sve rezervacije, aktivne sesije, prijavljene kvarove ili narudžbe. Korištenjem Iterator obrasca omogućava se prolazak kroz kolekcije bez poznavanja njihove interne strukture, čime se povećava fleksibilnost implementacije i pojednostavljuje pristup podacima.

## **MEDIATOR PATTERN**

Mediator obrazac mogao bi se koristiti za koordinaciju komunikacije između glavnih komponenti sistema. Trenutno su korisnici, rezervacije, sesije, uređaji i plaćanja međusobno povezani kroz veliki broj relacija. Uvođenjem centralne klase posrednika, na primjer GamingCentarMediator, komunikacija između ovih komponenti bila bi centralizovana. Kada korisnik kreira rezervaciju, posrednik bi bio odgovoran za provjeru dostupnosti uređaja, kreiranje rezervacije, pokretanje sesije i generisanje podataka za plaćanje. Na taj način smanjuje se međusobna zavisnost klasa i povećava preglednost sistema.